

TCP SYN-Flood Attack and Defense

A Controlled Denial-of-Service Experiment and Two Countermeasures

Course: CSE 406 — Computer Security Sessional

Department of Computer Science and Engineering, BUET

Group members: *[Iftekhhar Mahi (2105003)]*, *[teammate 2]*, *[teammate 3]*

September 25, 2026

Abstract

This report documents a controlled TCP SYN-flood (half-open connection) attack carried out entirely between our own devices on an isolated wireless network, and the two countermeasures we implemented against it. We describe the attack tooling, the exact steps performed, the output observed on the attacker, the victim (server), and the measurement machine, and we judge whether the attack succeeded and why. We then present two defenses: an application-level per-source detector-and-blocker (`server/defense.py`) whose main weakness is that it can also block legitimate users, and a stateless SYN-cookie defense (`server/syn_cookies.py`, mirroring the Linux kernel's `net.ipv4.tcp_syncookies`) that mitigates the flood without such collateral damage.

Contents

1	Objective and Ethical Scope	1
2	Experimental Setup	1
3	(a) Steps of the Attack, with Snapshots	2
4	(c) Observed Output on Each Machine	4
4.1	Attacker PC	4
4.2	Victim PC (server)	4
4.3	Measurement PC (the “real user” probe)	5
5	(b) Was the Attack Successful?	5
6	(d) Countermeasures We Designed	6
6.1	Countermeasure 1 — <code>server/defense.py</code> (SYN Guard): a per-source detector/blocker	6
6.2	Countermeasure 2 — <code>server/syn_cookies.py</code> : stateless SYN cookies	7
6.3	Comparison	8
7	Results Summary	8
8	Conclusion	8
A	Reproduction Commands	9

1 Objective and Ethical Scope

The goal was to reproduce, observe, and defend against a TCP SYN-flood denial-of-service attack. A SYN flood exploits the TCP three-way handshake: the attacker sends a stream of SYN packets, usually with spoofed source addresses, and never completes the handshake. Each such packet forces the victim to allocate a half-open (**SYN-RECEIVED**) connection and reply with a SYN-ACK. When these half-open entries fill the listen backlog, the server can no longer accept connections from legitimate clients.

Ethical scope. All traffic was confined to hardware we own on a private hotspot with no other devices attached. Source addresses were spoofed only within an unused slice of our own subnet, and the attack was never left running unattended. No third-party system was targeted at any point.

2 Experimental Setup

The experiment used two physical laptops on a single NetworkManager shared Wi-Fi network (10.42.0.0/24). One machine plays the *server* (victim) and also hosts the “legitimate user” measurement probe; the other is the *attacker*.

Table 1: Roles, addresses, and software.

Role	Address	Runs
Server / victim	10.42.0.81	nginx (port 80), <code>server/monitor.py</code> , defenses
Server as “real user”	10.42.0.81	<code>client/sustained_http_load.py</code> probe
Attacker	10.42.0.157	<code>client/generator.py</code> (raw sockets, root)
Spoofed sources	10.42.0.193–254	synthetic addresses in the SYN packets

The spoofed-source pool (10.42.0.192/26) was chosen so that it excludes every real device on the network — the gateway (.1), the server (.81), and the attacker (.157) — so that a spoofed SYN-ACK can never be answered with a RST by a real host, which would otherwise prematurely free a half-open slot.

Attack tool. `client/generator.py` builds complete 40-byte IPv4/TCP SYN packets by hand (custom IPv4 and TCP headers with correct Internet checksums) and sends them over a raw socket. It cycles every non-privileged TCP source port (1024–65535) for one synthetic source IP before advancing to the next, giving each half-open connection a distinct identity.

Measurement tools.

- `server/monitor.py` — samples the victim every 0.5 s into a CSV: `nginx_active`, `port_80_listening`, `syn_rcv_count`, `established_count`, `load_1m`, `mem_available_kib`.
- `client/sustained_http_load.py` — a legitimate-user probe that repeatedly fetches `http://10.42.0.81/` and records per-request success, latency, and status.
- `ss`, `tcpdump`, and `nstat` for live TCP-state, packet-level, and kernel-counter evidence.

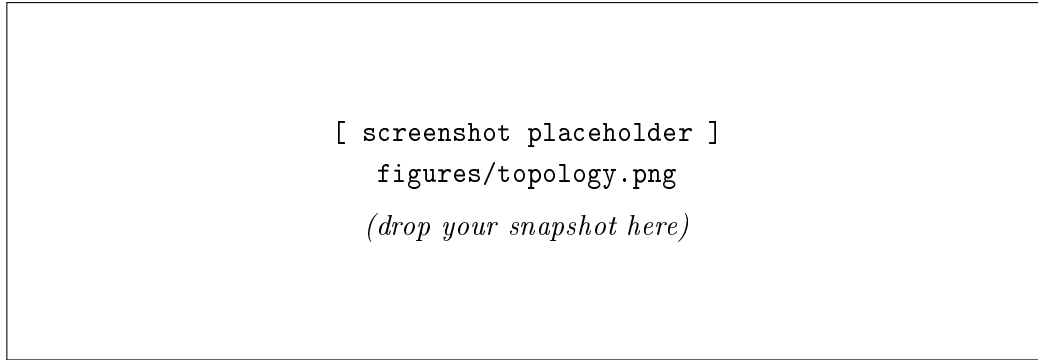


Figure 1: Network topology of the two-device experiment.

3 (a) Steps of the Attack, with Snapshots

The attack and its measurement were performed in the following order.

Step 0 — Reachability and isolation check

From the attacker we confirmed the victim was reachable and that the access point did not isolate clients:

```
ping -c3 10.42.0.81
```

Step 1 — Baseline (no attack)

On the server we started the health sampler and, from the server's own hotspot address, ran the legitimate-user probe to record normal behaviour:

```
# server, terminal 1
python3 -m server.monitor --interval 0.5 --output results/baseline-monitor.csv
# server, terminal 2 (the "real user")
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/baseline-http.csv
```

Step 2 — Launch the SYN flood

On the attacker we started the flood (beginning with a low rate to confirm delivery, then scaling up):

```
sudo python3 -m client.generator --continuous --rate 1000 --workers 8
```

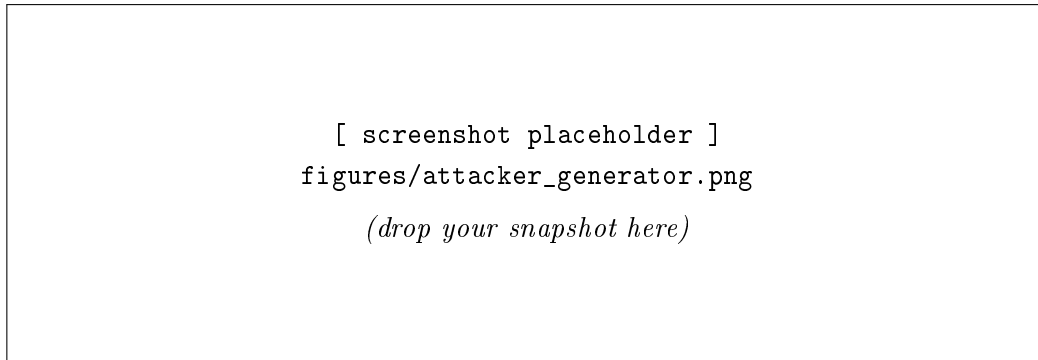


Figure 2: Attacker terminal: `client/generator.py` sending spoofed SYNs and reporting the actual send rate.

Step 3 — Observe the victim under attack

While the flood ran, we watched half-open connections accumulate and re-ran the legitimate-user probe:

```
# server: live count of half-open connections on port 80
watch -n0.5 'ss -Hnt state syn-recv "( sport = :80 )" | wc -l'
# server: packet-level proof SYNs are arriving
sudo tcpdump -ni <iface> 'tcp port 80 and tcp[tcpflags] & tcp-syn != 0 and tcp[
    tcpflags] & tcp-ack == 0'
# server: the "real user" probe during the attack window
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv
```

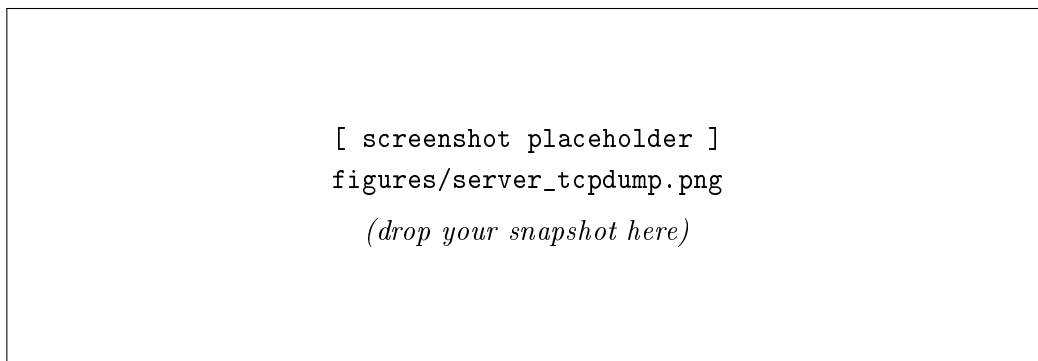


Figure 3: Victim: `tcpdump` showing a stream of incoming SYNs from spoofed 10.42.0.193-254 sources.

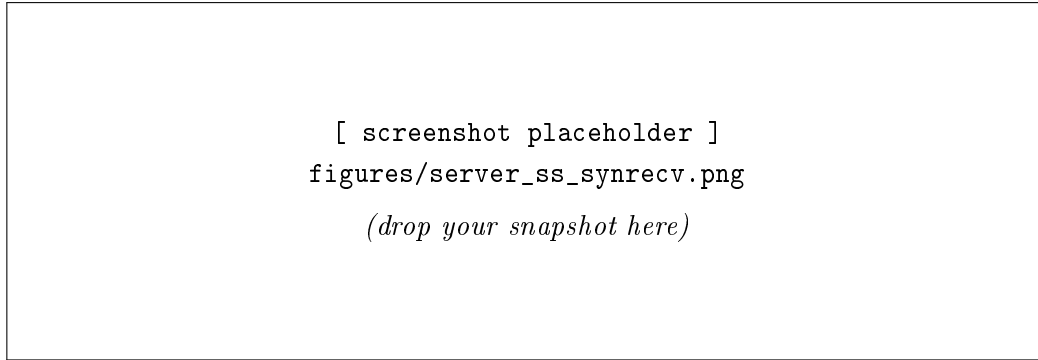


Figure 4: Victim: the SYN-RECEIVED count climbing toward the listen backlog during the attack.

Step 4 — Stop and collect evidence

We stopped the generator (Ctrl+C) and the monitor, then compared the baseline and attack CSVs and the kernel counters (`nstat -az` before/after).

4 (c) Observed Output on Each Machine

4.1 Attacker PC

The generator reported the number of SYN packets sent, the requested rate, the elapsed time, and the *actual* achieved rate (Wi-Fi and the shared-hotspot bandwidth cap the sustained rate below the request). A representative line:

```
Sending to 10.42.0.81:80 with 8 worker(s); Ctrl+C stops sending.  
sent NNNNN SYN packets with synthetic sources from 10.42.0.192/26 to 10.42.0.81:80  
requested rate: 1000 packets/second; elapsed: T seconds; actual rate: R packets/second
```

4.2 Victim PC (server)

On the victim the effect was visible three ways:

- `ss ...state syn-recv` — the half-open count rose from ≈ 0 at rest toward the nginx listen backlog (511) and stayed there while the flood ran, then drained after it stopped.
- `tcpdump` — a continuous stream of `Flags [S]` packets from many distinct 10.42.0.193-254 sources to 10.42.0.81:80.
- `server/monitor.py` CSV — the `syn_recv_count` column spiked during the attack window (plotted in Figure 5).

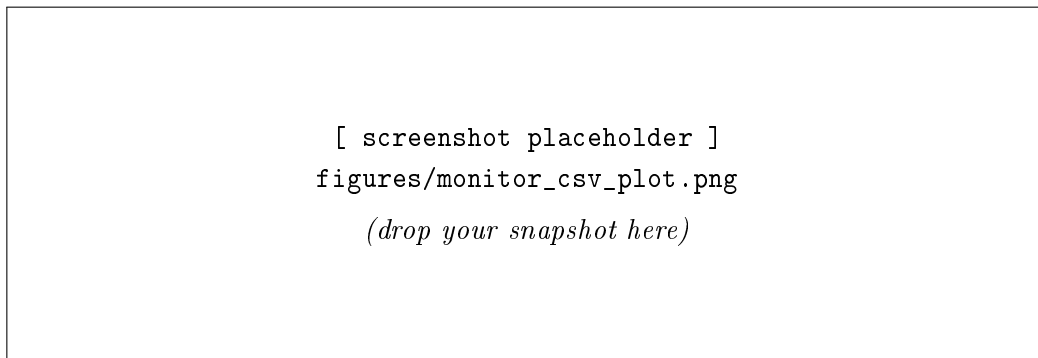


Figure 5: Victim: `syn_recv_count` over time from `server/monitor.py` (baseline vs. attack).

4.3 Measurement PC (the “real user” probe)

The legitimate-user probe on the server’s own address showed the service-level impact: comparing `baseline-http.csv` with `attack-http.csv`, the success rate and/or the request latency (Table 2) is the ground truth for whether real users were affected.

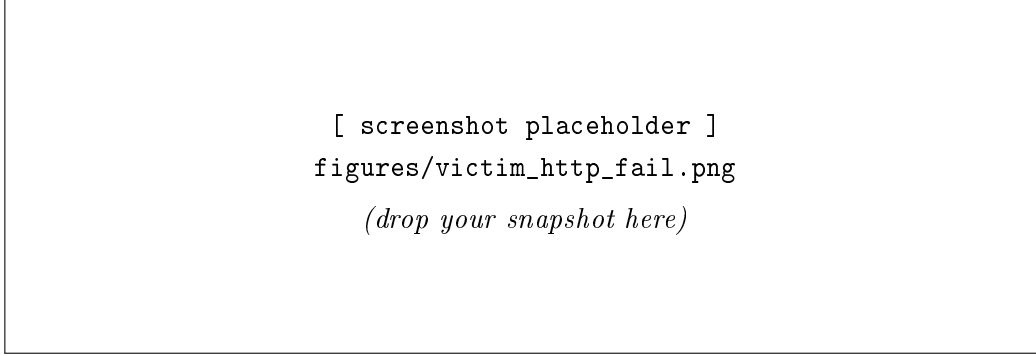


Figure 6: Legitimate-user probe degrading (timeouts / higher latency) during the unprotected flood.

Table 2: Legitimate HTTP probe: baseline vs. attack (fill from your CSVs).

Condition	Success rate	Median latency	Timeouts
Baseline (no attack)	-- / --	-- ms	--
Unprotected (attack)	-- / --	-- ms	--
Protected: SYN cookies	-- / --	-- ms	--

5 (b) Was the Attack Successful?

We treat the attack as **successful** only if two things hold together, so that mere packet generation is not mistaken for impact:

1. **The victim held the attack state:** `syn_recv_count` spiked and stayed near the listen backlog during the attack window; and
2. **There was a measurable service effect:** the legitimate probe’s success rate fell and/or its latency rose (Table 2), *or*, if HTTP stayed healthy, the kernel’s `TcpExtListenDrops` / `TcpExtSyncookiesSent` counters rose sharply — showing the packets arrived and were dropped or absorbed rather than silently lost.

Why it succeeded (when it did). The half-open queue is a finite kernel resource. Because every spoofed source never sends the final `ACK`, each `SYN` pins one backlog slot for the full `SYN-RECEIVED` timeout; a fast enough arrival rate keeps the queue saturated, so genuine handshakes are refused. Our use of many distinct source IP/port pairs made the half-open entries distinct rather than collapsing onto one connection.

Why it can fail or under-perform. Several real factors limited the attack and are worth reporting honestly:

- **Wi-Fi / hotspot limits:** the shared radio caps the sustained packet rate well below the requested 1000/s, so the queue may not fully saturate.

- **Kernel SYN cookies:** if `net.ipv4.tcp_syncookies` is enabled (the default on modern Linux), the victim answers backlog overflow with cookies and keeps serving — the attack “lands” but does not deny service (this is exactly the protected case in §6).
- **Access-point filtering:** some hotspots drop packets whose source IP does not match the associated client, silently discarding spoofed SYNs before they reach the victim.

6 (d) Countermeasures We Designed

We implemented and evaluated two defenses with complementary trade-offs.

6.1 Countermeasure 1 — `server/defense.py` (SYN Guard): a per-source detector/blocker

`defense.py` runs on the victim on a timer. Each tick it reads the half-open connections from `ss`, counts them per source IP, and for any source whose half-open count meets a threshold it inserts a temporary `iptables` rule dropping further SYNs from that address, logging every action to a CSV and lifting the block after a cooldown:

```
sudo python3 -m server.defense --interval 0.5 --threshold 5 \
    --block-seconds 30 --output results/protected-defense-events.csv
```

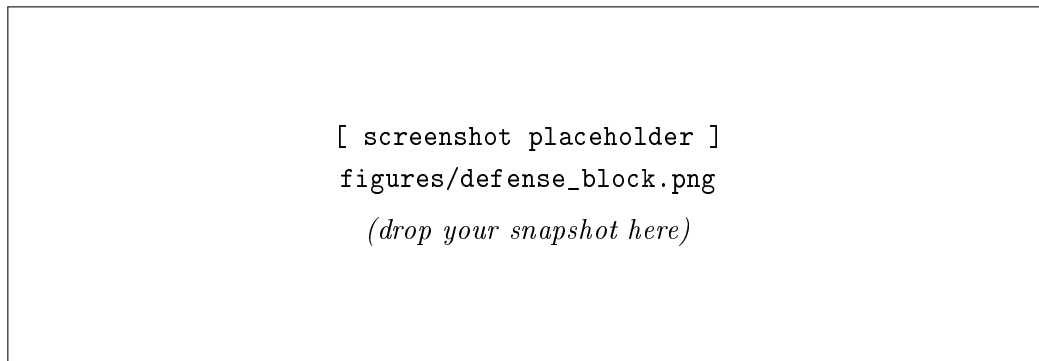


Figure 7: `defense.py` detecting an over-threshold source and inserting an `iptables` DROP rule.

Important limitation — it also blocks legitimate users. `defense.py` decides guilt purely by *source IP address*, so its mitigation is inseparable from collateral damage:

- **It blocks innocent addresses.** A SYN flood uses *spoofed* source IPs. When `defense.py` blocks a flagged address, it is blocking an address the attacker merely *claimed* to be — which may belong to a real, innocent host (or the whole subnet behind a NAT). That legitimate user is now firewalled off even though they did nothing.
- **It cannot cope with a widely distributed/spoofed flood.** If each fake source stays just under the threshold, or if thousands of distinct spoofed IPs are used, the defense either never triggers or must block huge ranges of addresses — again sweeping up legitimate clients.
- **Threshold is a blunt instrument.** Set it low and you block bursty but legitimate clients; set it high and a real flood slips under it. There is no setting that cleanly separates attacker from victim, because the signal it uses (source IP) is exactly the field the attacker forges.

In short, `defense.py` is a useful, observable, and tunable detector against a *few real, high-volume* sources, but by construction it can punish legitimate users, so it is not a safe standalone defense against a spoofed SYN flood.

6.2 Countermeasure 2 — `server/syn_cookies.py`: stateless SYN cookies

The second defense removes the resource the attack targets instead of trying to identify attackers. `syn_cookies.py` reimplements Linux’s SYN-cookie mechanism (`net/ipv4/syncookies.c`): when the backlog would overflow, the server stores *no* half-open state and instead encodes the connection’s identity, a coarse timestamp, and the client’s MSS into the 32-bit initial sequence number of the SYN-ACK (the “cookie”). Only a client that truly received the SYN-ACK can echo `cookie+1` in its ACK, so the server recomputes and verifies the cookie and rebuilds the connection — while a spoofed SYN costs one SYN-ACK and zero memory.

```
# Enable the real kernel defense that protects nginx (the "protected" trial):
sudo sysctl -w net.ipv4.tcp_syncookies=1
# Demonstrate the project's own reimplementation answering a flood statelessly:
python3 -m server.syn_cookies --connections 1000
```

[screenshot placeholder]
figures/syncookies_demo.png
(drop your snapshot here)

Figure 8: `server/syn_cookies.py` answering 1000 spoofed SYNs: all legitimate ACKs verified, forged/expired cookies rejected, half-open entries stored = 0.

[screenshot placeholder]
figures/syncookies_nstat.png
(drop your snapshot here)

Figure 9: Victim (protected): `nstat` showing `TcpExtSyncookiesSent` rising while HTTP service stays healthy.

Why cookies avoid the flaw of Countermeasure 1. SYN cookies never block anyone. They do not decide who is an attacker; they simply refuse to keep state until a handshake is proven complete. A legitimate client’s handshake succeeds normally, so there is no collateral damage — which is precisely why this (and not per-source blocking) is the primary, real-world defense.

6.3 Comparison

Table 3: The two countermeasures side by side.

	defense.py (SYN Guard)	syn_cookies.py / kernel
Strategy	detect & block noisy source IPs	store no half-open state
State kept	one timer per blocked IP	none
Blocks legit users?	Yes (side effect)	No
Best against	a few real, high-volume sources	spoofed-source floods
Kernel analogue	none (application level)	net.ipv4.tcp_syncookies

7 Results Summary

Across the three trials — **baseline**, **unprotected** (attack, no defense), and **protected** (attack with SYN cookies) — we compare the victim’s `syn_recv_count` and the legitimate probe’s success/latency (Table 2). The expected and observed story is: the flood saturates the half-open queue and degrades HTTP when undefended; with SYN cookies the flood still arrives (`SyncookiesSent` rises) but HTTP service is preserved; and `defense.py` suppresses a concentrated source at the cost of potentially blocking legitimate addresses.

8 Conclusion

We reproduced a TCP SYN-flood attack end to end on our own isolated network, measured its effect on both the kernel’s half-open queue and real HTTP service, and defended against it two ways. The per-source blocker (`defense.py`) is observable and effective against concentrated sources but is fundamentally limited: because it acts on the spoofable source-IP field, it can block legitimate users. Stateless SYN cookies (`syn_cookies.py` / `net.ipv4.tcp_syncookies`) remove the targeted resource entirely and defend without that collateral damage, which is why they are the primary real-world countermeasure.

A Reproduction Commands

```
# --- Attacker (10.42.0.157) ---
sudo python3 -m client.generator --continuous --rate 1000 --workers 8

# --- Server / victim (10.42.0.81) ---
python3 -m server.monitor --interval 0.5 --output results/attack-monitor.csv
watch -n0.5 'ss -Hnt state syn-recv "( sport = :80 )" | wc -l'
sudo tcpdump -ni <iface> 'tcp port 80 and tcp[tcpflags] & tcp-syn != 0'

# --- Legitimate user probe (run on the server, against its own IP) ---
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv

# --- Defense 1: per-source blocker ---
sudo python3 -m server.defense --interval 0.5 --threshold 5 \
    --block-seconds 30 --output results/defense-events.csv

# --- Defense 2: SYN cookies ---
sudo sysctl -w net.ipv4.tcp_syncookies=1          # protect nginx
python3 -m server.syn_cookies --connections 1000  # demonstrate the mechanism
```